

Dayanıklı Bir Mimari Arayışı: Taş-Kağıt-Makas Projesinin Evrimi

Sıfırdan kurulan bir modelden, Transfer Learning ile güçlendirilmiş gerçek zamanlı bir uygulamaya uzanan geliştirme yolculuğu.

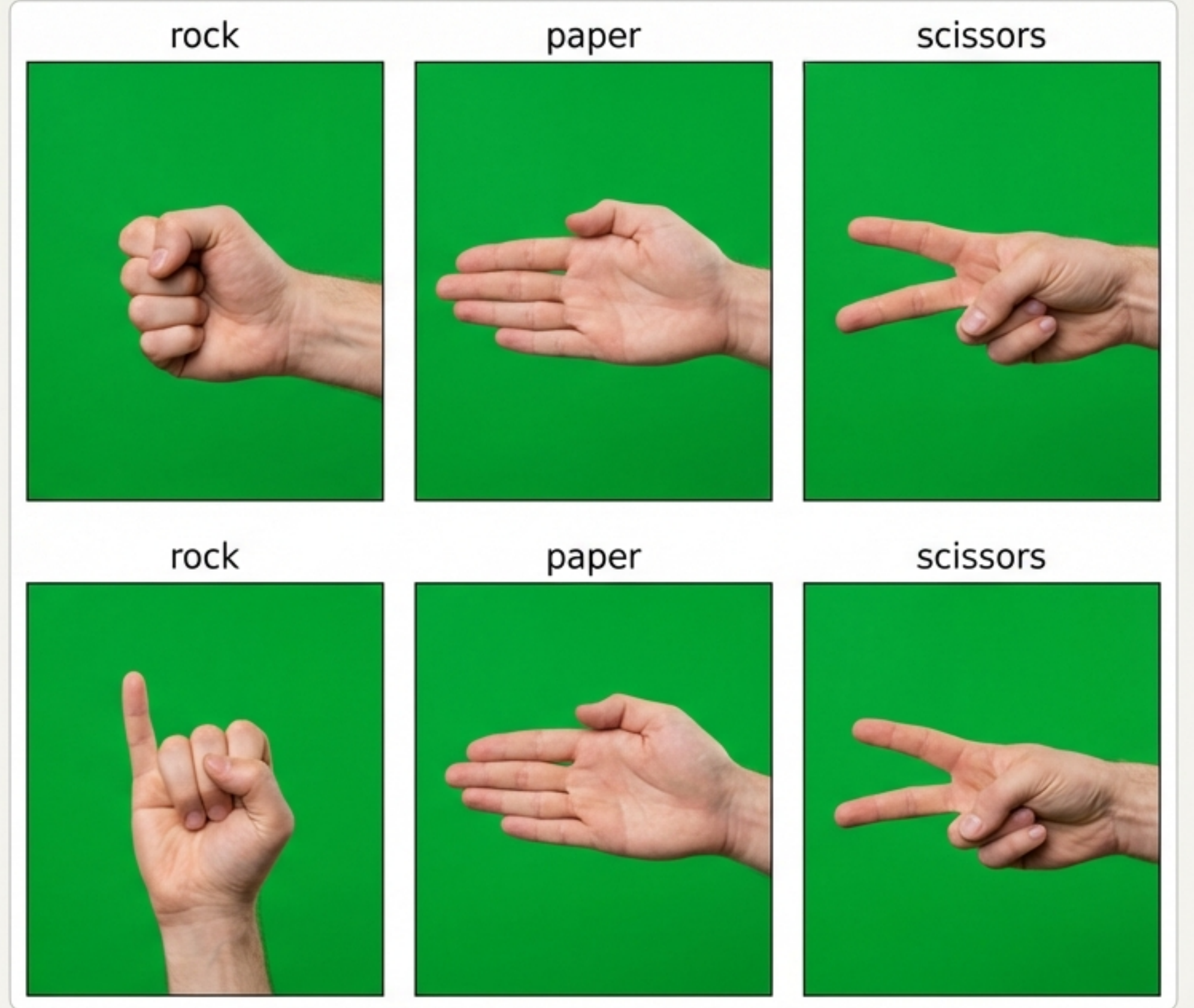


Her Şey Bir Veri Seti ile Başladı

Projenin temeli, Kaggle'dan temin edilen standart bir Taş-Kağıt-Makas veri setidir. Bu set, her biri farklı bir el hareketini temsil eden üç sınıftan oluşur. Görüntüler, yeşil bir arka plan önünde, tutarlı ışıklandırma koşullarında çekilmiştir. Bu, modelin temel özelliklerini öğrenmesi için ideal bir başlangıç noktasıdır.

 Eğitim Verisi: 1751 Görüntü

 Doğrulama Verisi: 437 Görüntü



İlk Deneme: Sıfırdan Bir Evrişimli Sinir Ağı (CNN)

İlk modelimiz, görüntü sınıflandırma için standart bir yaklaşım olan sıralı bir CNN mimarisi üzerine kurulmuştur. Model, temel özelliklerden (kenarlar, dokular) daha karmaşık desenlere doğru hiyerarşik bir öğrenme süreci için tasarlanmıştır.

```
# Modelimizi Sequential (sıralı) bir model olarak başlatıyoruz.
model = Sequential([
    # 1. EVRİŞİM BLOĞU
    # 32 filtre ile görüntüdeki temel özellikleri (kenarlar, köşeler) bulur.
    Conv2D(32, (3,3), activation='relu', input_shape=(150, 150, 3)),
    MaxPooling2D(2, 2),

    # 2. EVRİŞİM BLOĞU
    # Filtre sayısını 64'e çıkararak daha karmaşık desenleri öğrenir.
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2, 2),

    # 3. EVRİŞİM BLOĞU
    # Filtre sayısını 128'e çıkararak daha da karmaşık özellikleri öğrenir.
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2, 2),

    # KARAR VERME BLOĞU
    Flatten(),
    Dropout(0.5),
    Dense(512, activation='relu'),
    Dense(3, activation='softmax')
])
```

Model: "sequential"

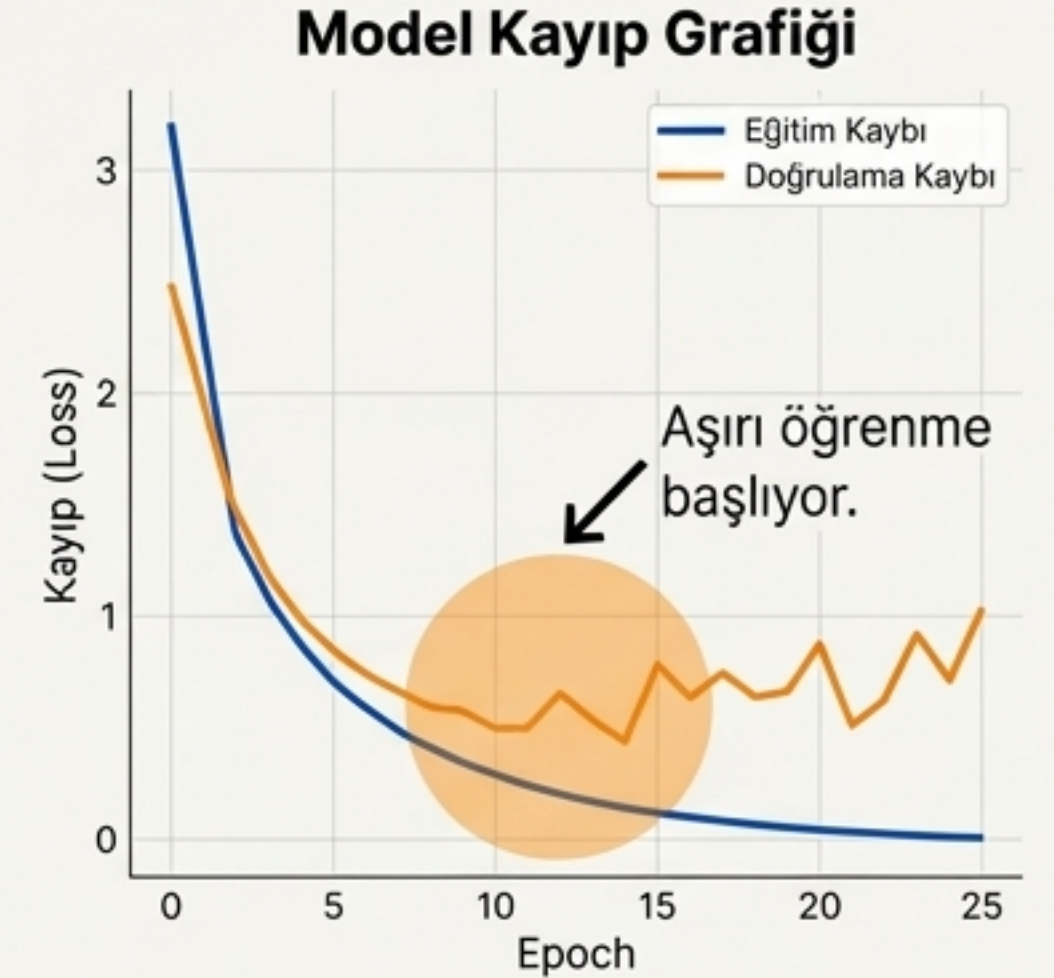
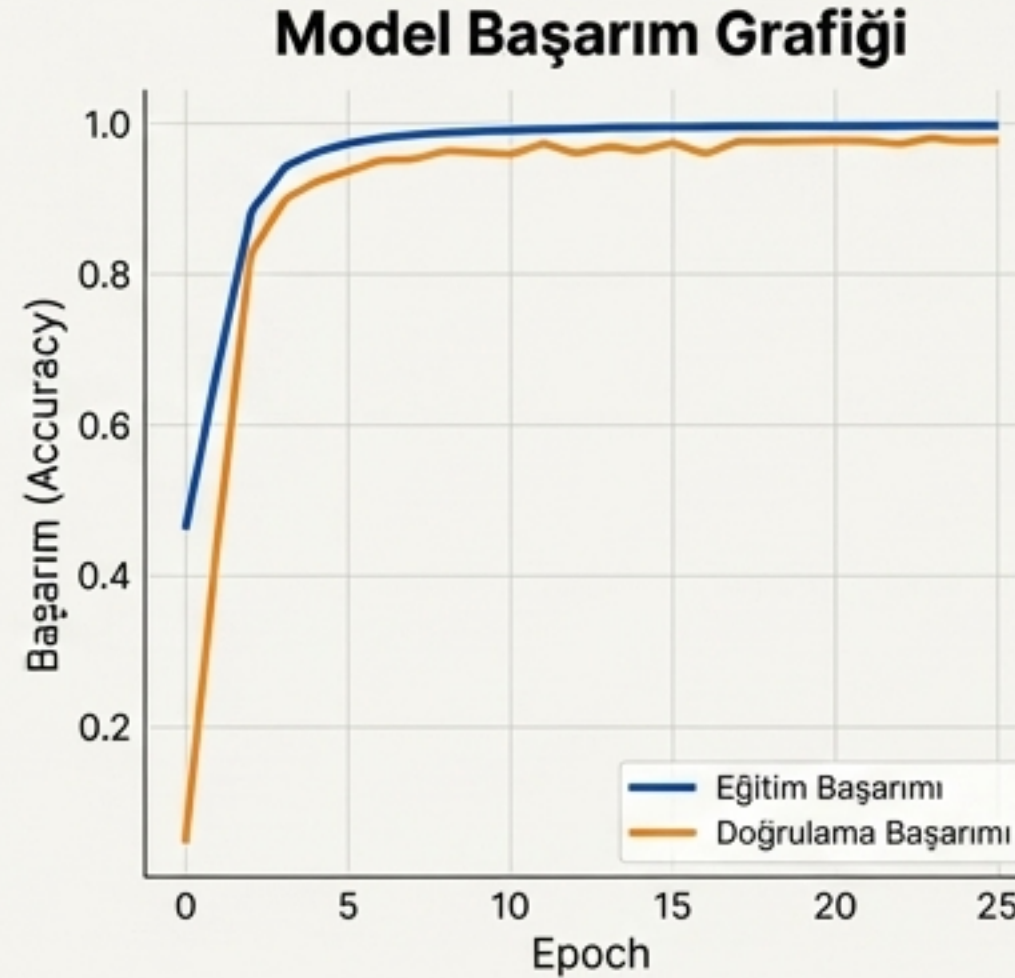
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
flatten (Flatten)	(None, 36992)	0
dropout (Dropout)	(None, 36992)	0
dense (Dense)	(None, 512)	18,940,416
dense_1 (Dense)	(None, 3)	1,539
Total params: 19,035,203		
Trainable params: 19,035,203		
Non-trainable params: 0		

Toplam 19 milyon
eğitilebilir parametre.

Yüksek Başarım, Gizli Bir Sorun: Aşırı Öğrenme

Modelin ilk eğitimi, eğitim verilerinde %99'un üzerinde bir başarıma ulaşarak oldukça başarılı görünüyor. Ancak, doğrulama verileri üzerindeki kayıp (validation loss) grafiği, modelin eğitim verilerini “ezberlediğini” ve yeni, görmediği verilere genelleme yapmakta zorlandığını gösteren bir dalgalanma sergiliyor. Bu durum, **aşırı öğrenmenin** (overfitting) tipik bir belirtisidir.

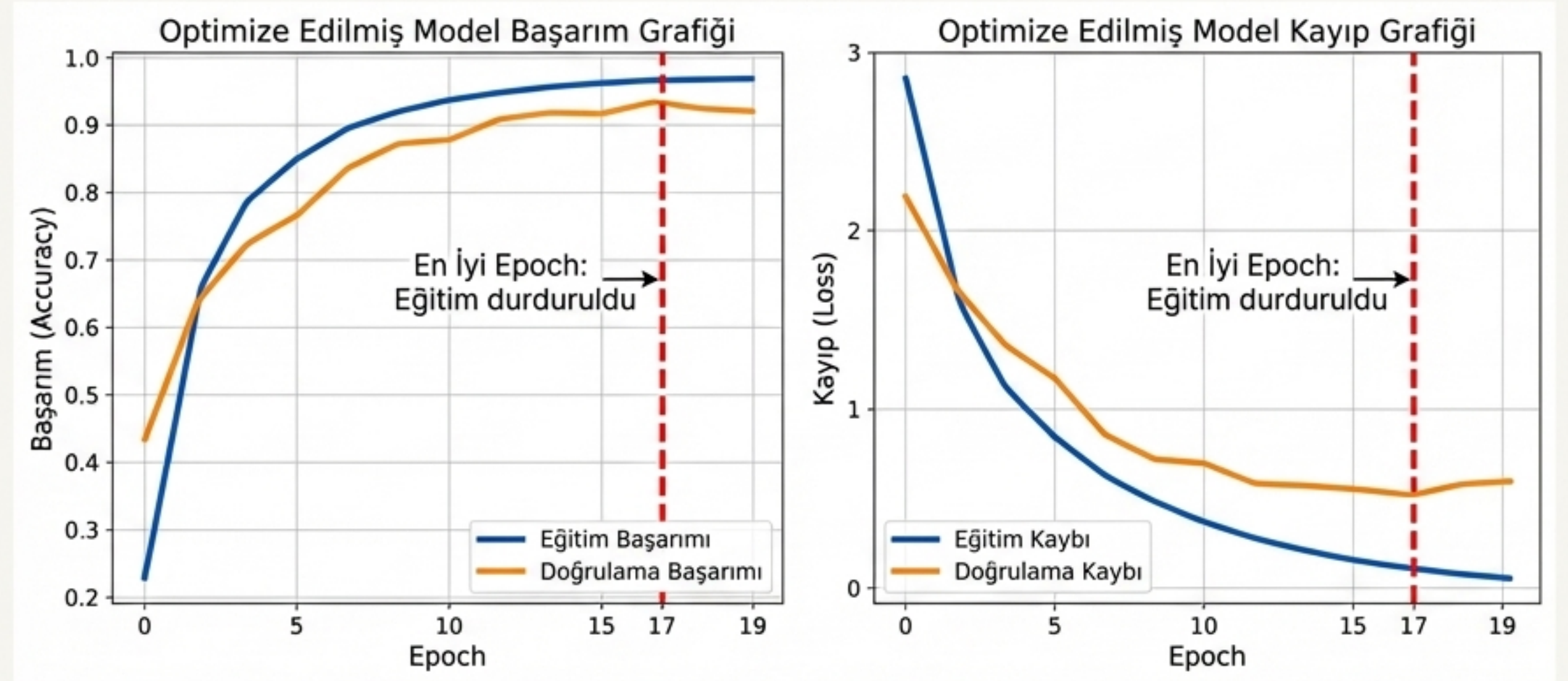
```
Epoch 1/20
55/55 - accuracy: 0.5255 - val_accuracy: 0.8970
...
Epoch 8/20
55/55 - accuracy: 0.9934 - val_accuracy: 0.9771
...
Epoch 18/20
55/55 - accuracy: 1.0000 - val_accuracy: 0.9817
...
```



İlk Optimizasyon: `EarlyStopping` ile İsrafı Önleme

`EarlyStopping` tekniği, modelin doğrulama kaybını izler ve performans artışı durduğunda eğitimi en iyi noktada otomatik olarak sonlandırır. Bu sayede hem gereksiz eğitimden kaçınılır hem de modelin en iyi performansa sahip sürümü korunur.

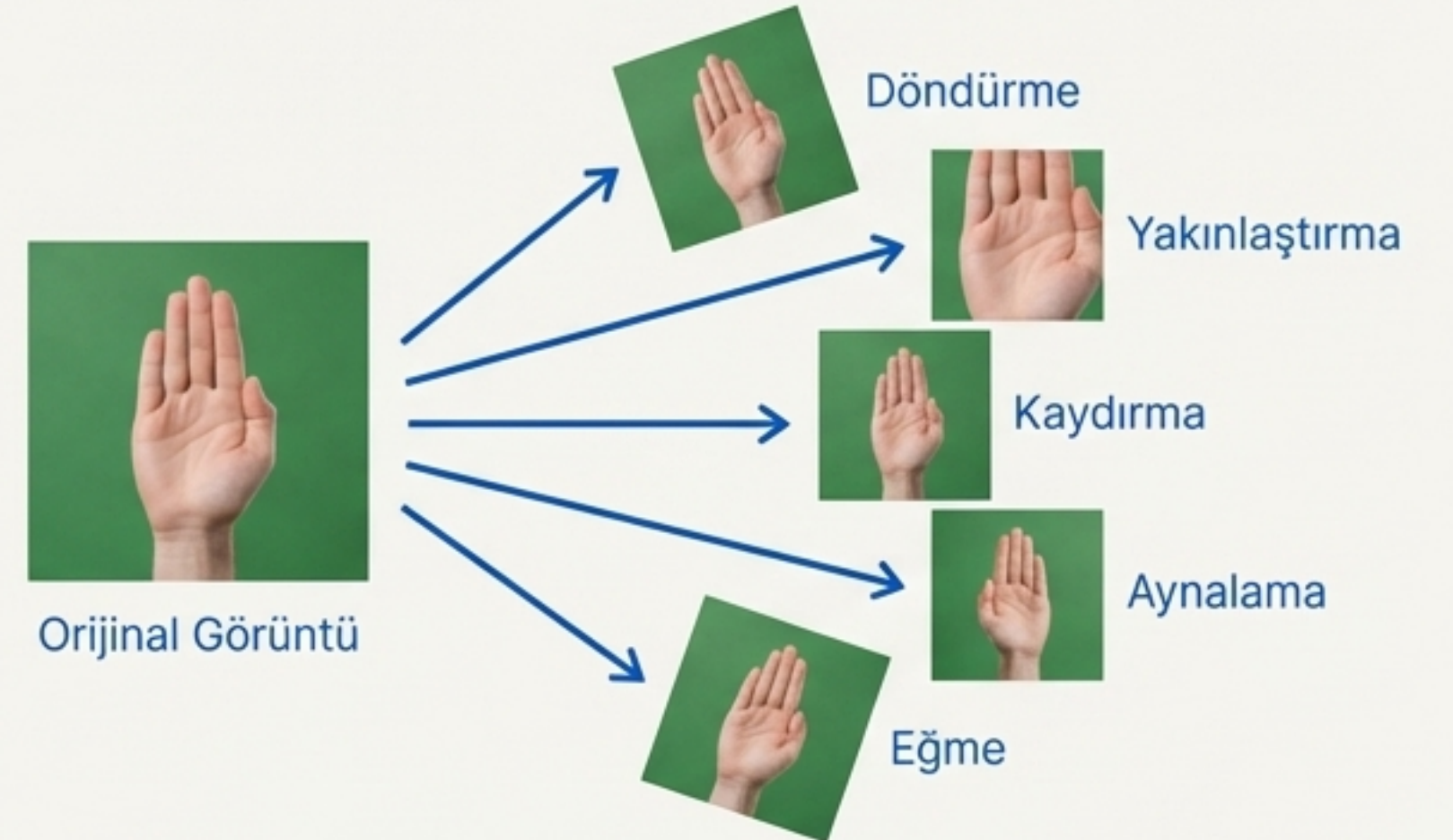
```
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```



Zorlukları Artırmak: Veri Artırma ile Genelleme Yeteneğini Güçlendirme

Modelimizin sadece temiz veri setinde değil, farklı açılardan ve pozisyonlardan gelen görüntülerde de başarılı olabilmesi için **Veri Artırma** tekniğini kullandık. Bu yöntem, mevcut eğitim görüntülerini rastgele dönüştürerek yapay olarak yeni ve çeşitli veriler üretir, modelin ezber yapmak yerine el hareketlerinin temel özelliklerini öğrenmesini sağlar.

```
train_datagen_augmented = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=20,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    fill_mode='nearest',  
    validation_split=0.2  
)
```



Gerçek Dünya Testi: Kontrol Dışı Veriler

Veri artırma, modelin genelleme yeteneğini artırsa da, orijinal veri setinin tek tip arka planı ve tutarlı koşulları hala bir sınırlamadır. Modelin gerçek dünya performansını ölçmek ve iyileştirmek için farklı arka planlar, ışık koşulları ve açılar içeren tamamen yeni bir veri seti oluşturuldu. Bu, modelin “konfor alanının” dışına çıkmasını sağlayan kritik bir adımdır.



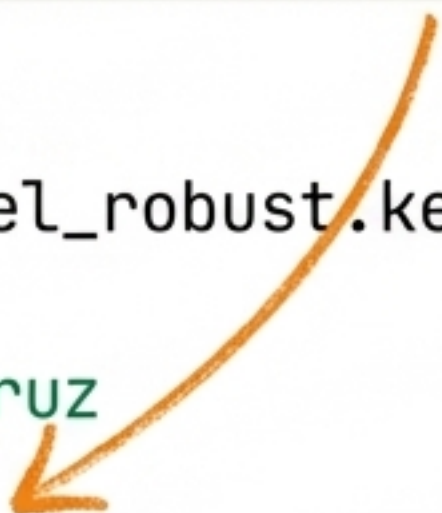
Yeni Koşullara Uyum: İnce Ayar (Fine-Tuning)

Daha önce eğittiğimiz sağlam (`robust`) modeli, yeni ve zorlu gerçek dünya verilerine adapte etmek için İnce Ayar tekniğini kullandık. Bu süreçte, model **çok düşük bir öğrenme oranı** ile yeni veriler üzerinde kısa bir süre daha eğitilir. Amaç, modelin daha önce öğrendiği temel bilgileri yok etmeden, sadece yeni veri setinin nüanslarını öğrenecek şekilde hassas bir şekilde ayarlanmasıdır.

Kritik Ayar: Öğrenilmiş bilgiyi korumak için.

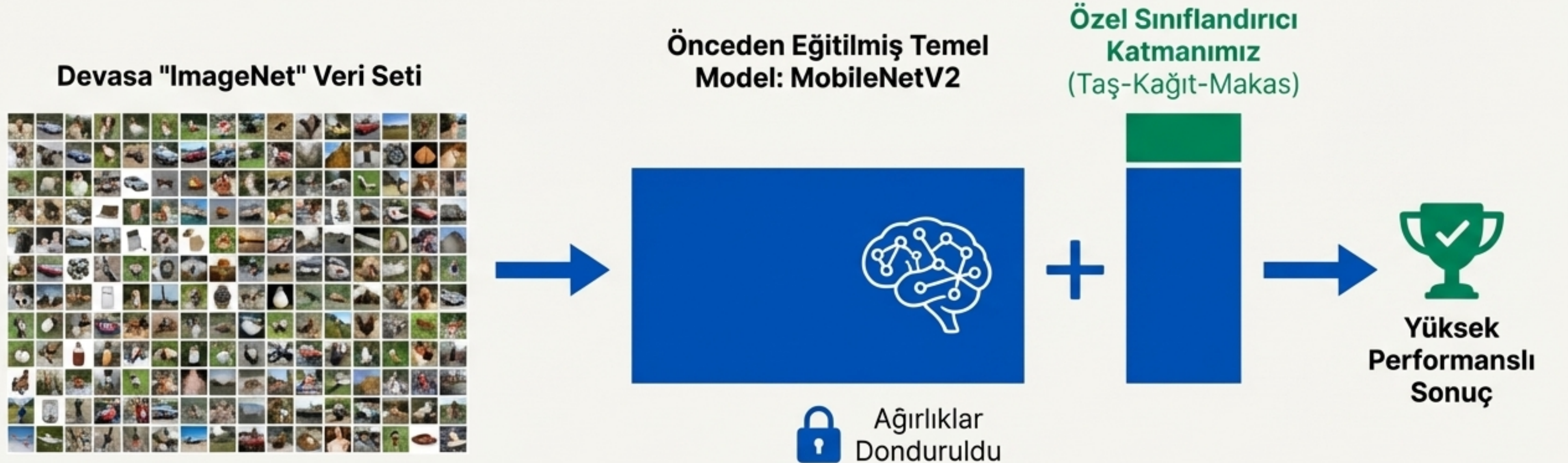
```
# En güçlü modelimizi yüklüyoruz
model_to_finetune = load_model('rps_model_robust.keras')

# Çok düşük bir öğrenme oranıyla derliyoruz
model_to_finetune.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```



Profesyonel Yaklaşım: Transfer Learning ile Devlerin Omuzlarında Yükselmek

Sıfırdan bir model eğitmek yerine, Google tarafından milyonlarca görüntü üzerinde eğitilmiş olan MobileNetV2 modelinin gücünden faydalandık. Bu yaklaşımda, MobileNetV2'nin önceden eğitilmiş katmanlarını bir "özellik çıkarıcı" olarak kullanır ve sadece kendi sınıflandırma problemimiz için üzerine küçük bir özel katman ekleyip eğitiriz. Bu, daha hızlı ve çok daha yüksek performanslı bir yöntemdir.



Nihai Sentez: Hibrit Veri Seti ve Agresif Artırma

En dayanıklı modeli oluşturmak için tüm veri kaynaklarımızı birleştirdik: Kaggle'dan gelen temiz ve büyük veri seti ile kendi ürettiğimiz zorlu ve gerçekçi veri seti. Bu 'Hibrit Veri Seti' üzerinde, modeli olası en zorlu koşullara hazırlamak için ışık parlaklığını değiştirmek gibi daha agresif veri artırma teknikleri uyguladık.



```
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=30,  
    ...  
    brightness_range=[0.8, 1.2], # Işık koşullarını değiştir  
    horizontal_flip=True,  
    ...  
)
```

İki Aşamalı Profesyonel Eğitim Stratejisi

Transfer Learning modelini eğitirken, maksimum performansı garantilemek için iki aşamalı bir strateji ve otomasyon araçları kullandık.

Aşamalı Eğitim



Aşama 1: Özellik Çıkarımı

Temel modelin (base_model) ağınlıkları dondurulur ve sadece bizim eklediğimiz yeni sınıflandırıcı katmanlar eğitilir.



Aşama 2: İnce Ayar (Fine-Tuning)

Temel modelin üst katmanları çözülür ve tüm model, çok düşük bir öğrenme oranıyla hassas bir şekilde ayarlanır.

Otomasyon ve Kontrol Callback'leri



`ModelCheckpoint`

Sadece en yüksek doğruluk oranına sahip modeli otomatik olarak kaydeder.



`EarlyStopping`

Performans artışı durduğunda eğitimi sonlandırır.



`ReduceLROnPlateau`

Gerekli olduğunda öğrenme oranını otomatik olarak düşürür.

Sonuç: Sınıfının En İyisi Performans

Transfer Learning, hibrit veri seti, agresif veri artırma ve profesyonel eğitim stratejisinin birleşimi, projenin en doğru ve en dayanıklı modeli olan `rps\_model\_pro\_best.keras`'ı ortaya çıkardı. Eğitim konsolu çıktıları, modelin doğrulama verileri üzerinde sürekli olarak yüksek başarımlar elde ettiğini ve en iyi versiyonunun 'ModelCheckpoint' tarafından başarıyla kaydedildiğini göstermektedir.

```
--- AŞAMA 1: Özellik Çıkarımı Eğitimi Başlatılıyor... ---
```

```
Epoch 1/10
```

```
66/66 ... val_accuracy: 0.9677
```

```
Epoch 1: val_accuracy improved from -inf to 0.96774, saving model to /content/drive/My.../rps_model_pro_best.keras
```

```
Epoch 2/10
```

```
66/66 ... val_accuracy: 0.9772
```

```
Epoch 2: val_accuracy improved from 0.96774 to 0.97723, saving model to /content/drive/My.../rps_model_pro_best.keras
```

```
Epoch 3/10
```

```
66/66 ... val_accuracy: 0.9810
```

```
Epoch 3: val_accuracy improved from 0.97723 to 0.98102, saving model to /content/drive/My.../rps_model_pro_best.keras
```

```
Epoch 4/10
```

```
66/66 ... val_accuracy: 0.9810
```

```
Epoch 4: val_accuracy did not improve from 0.98102
```

```
...
```

Modeli Hayata Geçirme: Gerçek Zamanlı Uygulama Mimarisi

Eğitilen `rps\_model\_pro\_best.keras` modeli, gerçek zamanlı bir oyun uygulamasına entegre edildi. Bu süreçte iki ana teknoloji kullanıldı:

1. **OpenCV**: Kameradan canlı video akışını almak için kullanılır.
2. **MediaPipe**: Google'ın bu kütüphanesi, video akışındaki elleri anında tespit eder. Tespit edilen el bölgesi, analiz için modelimize gönderilir.

Kurulum

```
MODEL_PATH = 'rps_model_pro_best.keras'
CLASS_NAMES = ['paper', 'rock', 'scissors']

# --- Model ve MediaPipe Kurulumu ---
try:
    model = tf.keras.models.load_model(MODEL_PATH)
    cap = cv2.VideoCapture(0)
    mp_hands = mp.solutions.hands
    hands1 = mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7)

    # ...
except Exception as e:
    # ...
```

Oyun Döngüsü

```
# --- Ana Oyun Döngüsü ---
while True:
    ret, frame = cap.read()
    # ...
    roi1 = frame[100:400, 50:350]

    # Oyuncu 1
    processed_hand1, landmarks1 = process_hand(roi1, hands1)

    if processed_hand1 is not None:
        prediction1 = model.predict(processed_hand1, verbose=0)
        move1 = CLASS_NAMES[np.argmax(prediction1)]

    # ...
```

Nihai Ürün: Profesyonel Taş-Kağıt-Makas



Bu arayüz, eğitilen yapay zeka modelinin MediaPipe ve OpenCV ile birleşerek nasıl interaktif ve akıcı bir kullanıcı deneyimi yarattığını göstermektedir.

Bir Modelin Evrimi: Yolculuktan Çıkarılan Dersler

Sıfırdan kurulan basit bir CNN modelinden, artan zorluklara karşı sürekli iyileştirme ve adaptasyonla geliştirilen, **Transfer Learning** tabanlı, gerçek zamanlı ve dayanıklı bir Taş-Kağıt-Makas oyununa. Bu proje, modern yapay zeka geliştirmede en iyi sonuçlara ulaşmanın, tek bir mükemmel model inşa etmekten ziyade, sürekli test etme, sorunu doğru teşhis etme ve doğru tekniği doğru yerde uygulama üzerine kurulu bir evrim süreci olduğunu göstermektedir.

